



C++ TIOBE comeback

Pushed out of the top three by Python last September, C++ has returned to 3rd position.
<https://dgit.in/may19-3>



Atom 1.36 speeds fuzzy find

Update has better performance with fuzzy find and smarter editing of individual files.
<https://dgit.in/may19-4>

THE LISTENER

To attain this, the directions are randomly changed at any instant on comparing a randomly generated number with the probability of change in direction of an invader along x and y axes. At any instant, the movement of the invader is also a factor that needs to be considered. An invader may keep moving without any hindrances and when it hits a wall, it needs to bounce back and that too has been handled by the code appropriately.

A point to note is that this function `event_step` has not been called

```
def event_step(self, time_passed, delta_mult):
    self.fitness += 1
    # Change directions
    if random.random() <= self.attributes.get('x_prob_change'):
        self.xvelocity = -self.xvelocity
    if random.random() <= self.attributes.get('y_prob_change'):
        self.yvelocity = -self.yvelocity

    # Bouncing off the edges and the wall
    if self.bbox.left < 0:
        self.bbox.left = 0
        self.xvelocity = abs(self.xvelocity)
    elif self.bbox.right > sge.game.current_room.width:
        self.bbox.right = sge.game.current_room.width
        self.xvelocity = -abs(self.xvelocity)
```

A part of the `event_step` function

from any other method or function. It behaves like a listener, establishing a channel which is always on the look for changes of any kind to the invader. Should any such change be encountered, this function is automatically

called and the fitness is increased by one following the other changes. To make it transparent, we understand that at any moment there lies a possibility for the invader to bounce at a wall and/or change its motion for whatever reason. And the longer the invader stays in the particular game, the more fit it is.

Like we have set the initial properties for the invaders, we do the same for the players. We set the basic movements (left and right) along with the start position. Again, with the help of the function `event_step` we capture the keypress (whether the player is to be moved left or right), find its speed and further animate the sprite to move in accordance with the direction of the keypress. Interesting, isn't it?

There is also a function that is going to take care of the shooting by the player for which the only prerequisite is that the number of invaders should be greater than the minimum and the

number of bullets should be lower than the minimum.

We've also defined a class for the bullet wherein we set the sprite animation for the bullet and the velocity of the bullet at its initiation.

```
class InvadersGame(sge.dap.Game):
    # Evolution constants
    RESX = 960
    RESY = 540
    # Object's position
    PLAYER_YOFFSET = 50
    PLAYER_SPEED = 4
    BULLET_START_SPEED = 20
    WALL_YOFFSET = 70
    WALL_HEIGHT = 2
    # Number of ms between generations (if it reaches a lower limit)
    GENERATION_TIME = 5000
    MIN_GEN_TIME = 2000
    # Colors
    CITIUS_COLOR = sge.gfx.Color('#EF7D10')
    IMMUNIT_COLOR = sge.gfx.Color('#15AFF0')
    # Minimum number of invaders that must survive
    MIN_MIN = 4
    # Number of invaders to lose the game
    MAX_MIN = 100
```

Setting up the initial game properties

In the `event_step` function for the bullet, we deal with two possibilities, one of which is the destruction of the bullet itself, i.e., the bullet has been fired by the player but it has failed to hit any target invader and has hit the boundaries. The other case is when the invader has been directly hit by the bullet and the invader count has been reduced by one.

Next up, we understand what the game class does. We import the game engines, the participants and evolutionary algorithm in the game class and define some global variables to handle the player speed, bullet speed,

*CODING MATTERS



*Mozilla WebThings beyond experimental

The Mozilla IoT team announced that after two years of development and multiple updates, Project Things is now known as Mozilla WebThings.
<https://dgit.in/MzilaWT>



*Red Hat takes over OpenJDK maintenance

Red Hat will look after aspects such as security patches and bug fixes for the OpenJDK 8 and 11, taking over from Oracle. Updates may still come from either.
<https://dgit.in/RHAtJDK>



*Whirl adds local AirFlow development

A test environment for Apache Airflow that lets you carry out local development and testing of Apache Airflow workflows has been made available as open source.
<https://dgit.in/Whirl>